

MODUL 10

Nama: Ryan Akeyla Novianto Widodo

Kelas: 12 IF 07

NIM : 103112400081

DASAR TEORI

Dasar Teori C++

Diambil dari internet dan dirangkum ulang oleh saya dan dibantu dengan AI.

1. Inti Program

- Syntax: Mirip bahasa Inggris, tapi huruf besar dan kecil beda arti. Setiap perintah biasanya diakhiri dengan titik koma (;).

- Komentar: Catatan di kode yang tidak dijalankan.

- Satu baris: // Ini komentar

- Banyak baris: /* Ini komentar yang panjang */

- Header: File yang berisi daftar perintah dan fungsi yang bisa dipakai. Contoh:

cpp

```
#include <iostream> // Untuk perintah input/output (cout, cin)
```

- main() : Fungsi utama, tempat program mulai berjalan.

cpp

```
int main() {
```

```
    // Kode program di sini
```

```
    return 0; // Menandakan program selesai dengan benar
```

```
}
```

- Namespace: Seperti "area" untuk nama-nama perintah. std adalah namespace standar C++.

cpp

```
using namespace std; // Supaya tidak perlu tulis std::cout, std::cin, dll.
```

- Input/Output:

- cout : Menampilkan sesuatu di layar.

- cin : Membaca input dari keyboard.

cpp

```
cout << "Halo!" << endl; // Menampilkan "Halo!"
```

```
int umur;
```

```
cin >> umur; // Membaca angka dari keyboard dan disimpan di variabel "umur"
```

2. Jenis Data

- Integer (Bilangan Bulat):

- int : Bilangan bulat (contoh: -10, 0, 25).

- short : Bilangan bulat pendek.

- long : Bilangan bulat panjang.

- Floating-Point (Bilangan Desimal):

- float : Bilangan desimal (contoh: 3.14).

- double : Bilangan desimal dengan ketelitian lebih tinggi.

- Character (Karakter):

- char : Satu huruf, angka, atau simbol (contoh: 'a', 'Z', '5').

- Boolean (Logika):

- bool : Nilai benar (true) atau salah (false).

- void : "Kosong". Dipakai untuk fungsi yang tidak menghasilkan nilai.

3. Variabel

- Deklarasi: Memberi tahu program nama dan jenis data variabel.

cpp

```
int umur; // Variabel bernama "umur" untuk menyimpan bilangan bulat
```

```
double harga; // Variabel bernama "harga" untuk menyimpan bilangan desimal
```

- Inisialisasi: Memberi nilai awal ke variabel.

cpp

```
int umur = 20; // "umur" diisi dengan nilai 20
```

```
double harga = 99.50; // "harga" diisi dengan nilai 99.50
```

4. Operator

- Aritmatika: + (tambah), - (kurang), * (kali), / (bagi), % (sisa bagi).
- Penugasan: = (mengisi nilai ke variabel).
- Perbandingan: == (sama dengan), != (tidak sama dengan), > (lebih besar), < (lebih kecil), >= (lebih besar atau sama dengan), <= (lebih kecil atau sama dengan).
- Logika: && (DAN), || (ATAU), ! (TIDAK).
- Increment/Decrement: ++ (tambah 1), -- (kurang 1).

5. Struktur Kontrol

- if : Melakukan sesuatu jika kondisi benar.

cpp

```
if (umur >= 17) {  
    cout << "Anda sudah dewasa." << endl;  
} else {  
    cout << "Anda belum dewasa." << endl;  
}
```

- switch : Memilih salah satu dari beberapa pilihan.

cpp

```
int hari = 3;  
switch (hari) {  
    case 1:  
        cout << "Senin" << endl;  
        break;  
    case 2:
```

```

        cout << "Selasa" << endl;

        break;

    case 3:

        cout << "Rabu" << endl;

        break;

    default:

        cout << "Hari lain" << endl;

}

```

- for : Melakukan sesuatu berulang kali sebanyak yang ditentukan.

cpp

```

for (int i = 0; i < 5; i++) {

    cout << i << " "; // Menampilkan angka 0 sampai 4

}

cout << endl;

```

- while : Melakukan sesuatu berulang kali selama kondisi benar.

cpp

```

int angka = 1;

while (angka <= 3) {

    cout << angka << " ";

    angka++;

}

cout << endl;

```

- do-while : Sama seperti while , tapi dilakukan minimal sekali.

cpp

```

int pilihan;

do {

    cout << "Pilih 1, 2, atau 3: ";

```

```
cin >> pilihan;  
} while (pilihan < 1 || pilihan > 3);
```

6. Fungsi

- Definisi: Sekumpulan kode yang melakukan tugas tertentu.

cpp

```
int tambah(int a, int b) {  
    return a + b;  
}
```

- Pemanggilan: Menggunakan fungsi.

cpp

```
int hasil = tambah(5, 2); // Memanggil fungsi "tambah" dengan angka 5 dan 2  
cout << "Hasil: " << hasil << endl; // Menampilkan "Hasil: 7"
```

7. Pointer

- Definisi: Variabel yang menyimpan alamat memori variabel lain. (Agak rumit, tapi penting).

cpp

```
int umur = 25;  
  
int *p_umur = &umur; // p_umur menyimpan alamat memori dari variabel umur
```

DASAR TEORI TREE

Pengertian Stack

Tree adalah struktur data **non-linear** yang digunakan untuk merepresentasikan data secara **hierarkis**. Tree terdiri dari sekumpulan **node (simpul)** yang saling terhubung, dengan satu node utama yang disebut **root**. Setiap node dapat memiliki **child** dan **parent**, serta tidak mengandung **siklus**.

Beberapa istilah penting dalam tree antara lain **root**, **parent**, **child**, **sibling**, **leaf**, **level**, dan **height**. Berdasarkan strukturnya, tree dibagi menjadi **general tree**, **binary tree**, dan **binary search tree (BST)**, di mana BST memiliki aturan nilai child kiri lebih kecil dan child kanan lebih besar dari parent.

Dalam C++, tree direpresentasikan menggunakan **struct dan pointer**. Operasi dasar pada tree meliputi **insert**, **search**, dan **traversal**. Traversal bertujuan untuk mengunjungi seluruh node dan terdiri dari **preorder**, **inorder**, dan **postorder**.

Secara umum, tree sangat efektif untuk mengelola data yang bersifat bertingkat, seperti struktur folder, database, dan sistem pencarian, meskipun implementasinya lebih kompleks dibandingkan struktur data linear.

Guided 1

```
#include <iostream>
using namespace std;

struct Node
{
    int data;
    Node *kiri, *kanan;
};

Node *buatNode(int nilai)
{
    Node *baru = new Node;
    baru->data = nilai;
    baru->kiri = baru->kanan = NULL;
    return baru;
}

// 1. INSERT (menyisipkan data)
Node *insert(Node *root, int nilai) {
    if (root == NULL) {
        return buatNode(nilai);
    }

    if (nilai < root->data) {
        root->kiri = insert(root->kiri, nilai);
    } else {
        root->kanan = insert(root->kanan, nilai);
    }

    return root;
};

Node *search(Node *root, int nilai) {
    if (root == NULL || root->data == nilai) {
        return root;
    }

    if (nilai < root->data) {
        return search(root->kiri, nilai);
    }

    return search(root->kanan, nilai);
}

// Helper: Cari nilai terkecil (untuk proses hapus)
Node *nilaiTerkecil(Node *node) {
    Node *current = node;
    while (current && current->kiri != NULL) {
```

```

    current = current->kiri;
}
return current;
}

// 3. DELETE (menghapus data - diperlukan untuk update)
Node *hapus(Node *root, int nilai) {
    if (root == NULL) {
        return root;
    }

    if (nilai < root->data) {
        root->kiri = hapus(root->kiri, nilai);
    } else if (nilai > root->data) {
        root->kanan = hapus(root->kanan, nilai);
    } else {
        // Jika data ketemu
        if (root->kiri == NULL) {
            Node *temp = root->kanan;
            delete root;
            return temp;
        }
        else if (root->kanan == NULL) {
            Node *temp = root->kiri;
            delete root;
            return temp;
        }
        // Jika punya 2 child: ambil terkecil dari kanan
        Node*temp = nilaiTerkecil(root->kanan);
        root->data = temp->data;
        root->kanan = hapus(root->kanan, temp->data);
    }
    return root;
}

// 4. UPDATE (ubah data)
Node *update(Node *root, int lama, int baru) {
    if (search(root, lama) != NULL) {
        root = hapus(root, lama);
        root = insert(root, baru);
        cout << "Data " << lama << " diupdate menjadi " << baru << endl;
    } else {
        cout << "Data " << lama << " tidak ditemukan!" << endl;
    }
    return root;
}

// 5. TRAVERSAL (menampilkan data)
void preOrder(Node *root) {
    // Akar -> kiri -> kanan

```

```

    if (root != NULL) {
        cout << root->data << " ";
        preOrder(root->kiri);
        preOrder(root->kanan);
    }
}

void inOrder(Node *root) {
    // kiri -> akar -> kanan
    if (root != NULL) {
        inOrder(root->kiri);
        cout << root->data << " ";
        inOrder(root->kanan);
    }
}

void postOrder(Node *root) {
    // kiri -> kanan -> akar
    if (root != NULL) {
        postOrder(root->kiri);
        postOrder(root->kanan);
        cout << root->data << " ";
    }
}

int main() {
    Node *root = NULL;

    cout << "=== 1. INSERT DATA ===" << endl;
    root = insert(root, 10);
    insert(root, 5);
    insert(root, 20);
    insert(root, 3);
    insert(root, 7);
    insert(root, 15);
    insert(root, 25);
    cout << "Data berhasil dimasukkan.\n" << endl;

    cout << "=== 2. TAMPILKAN TREE (TRAVERSAL) ===" << endl;
    cout << "PreOrder  : ";
    preOrder(root);
    cout << endl;
    cout << "InOrder   : ";
    inOrder(root);
    cout << endl;
    cout << "PostOrder : ";
    postOrder(root);
    cout << endl << endl;

    cout << "=== 3. TEST SEARCH ===" << endl;

```

```

int cari1 = 7, cari2 = 30;
cout << "Cari " << cari1 << ": " << (search(root, cari1) ? "Ketemu" : "Tidak ada")
<< endl;
cout << "Cari " << cari2 << ": " << (search(root, cari2) ? "Ketemu" : "Tidak ada")
<< endl;
cout << endl;

cout << "=== 4. TEST UPDATE ===" << endl;
// mengubah angka 5 menjadi 8
root = update(root, 5, 8);
cout << "Hasil InOrder setelah update: ";
cout << endl;
cout << endl;

cout << "PreOrder  : ";
preOrder(root);
cout << endl;
cout << "InOrder   : ";
inOrder(root);
cout << endl;
cout << "PostOrder  : ";
postOrder(root);
cout << endl << endl;

cout << "=== 5. TEST DELETE ===" << endl;
// menghapus angka 20 (Node yang punya child)
cout << "Menghapus angka 20..." << endl;
root = hapus(root, 20);

cout << "PreOrder  : ";
preOrder(root);
cout << endl;
cout << "InOrder   : ";
inOrder(root);
cout << endl;
cout << "PostOrder  : ";
postOrder(root);
cout << endl << endl;

return 0;
}

```

Hasil Coding:

```

c:\Ryan\Semester 3\IF 07\Struktur Data\MODUL10_103112400081_StrukturData\output>.\"guided1.exe"
=== 1. INSERT DATA ===
Data berhasil dimasukkan.

=== 2. TAMPILKAN TREE (TRAVERSAL) ===
PreOrder : 10 5 3 7 20 15 25
InOrder : 3 5 7 10 15 20 25
PostOrder : 3 7 5 15 25 20 10

=== 3. TEST SEARCH ===
Cari 7: Ketemu
Cari 30: Tidak ada

=== 4. TEST UPDATE ===
Data 5 diupdate menjadi 8
Hasil InOrder setelah update:

PreOrder : 10 7 3 8 20 15 25
InOrder : 3 7 8 10 15 20 25
PostOrder : 3 8 7 15 25 20 10

=== 5. TEST DELETE ===
Menghapus angka 20...
PreOrder : 10 7 3 8 25 15
InOrder : 3 7 8 10 15 25
PostOrder : 3 8 7 15 25 10

```

Tujuan program ini adalah membuat program dari bahasa c++ yang mana kita membuat dan mengimplementasikan struktur data Tree, khususnya Binary Search Tree (BST), menggunakan bahasa pemrograman C++. Program ini dirancang untuk mengelola data bertipe integer dengan menerapkan operasi-operasi dasar pada BST, yaitu insert (penyisipan data), search (pencarian data), delete (penghapusan data), update (perubahan data), serta traversal (penelusuran node) yang meliputi preorder, inorder, dan postorder.

Selain itu, program ini bertujuan untuk memahami cara kerja BST secara rekursif, di mana setiap data yang dimasukkan akan ditempatkan sesuai dengan aturan BST (data lebih kecil berada di sebelah kiri dan data lebih besar berada di sebelah kanan). Dengan adanya traversal inorder, program juga dapat menampilkan data dalam urutan terurut secara menaik, sehingga memudahkan pengguna dalam melihat struktur dan isi tree.

Unguided

Coding:

```
#include <iostream>
using namespace std;

typedef int infotype;

struct Node {
    infotype info;
    Node* left;
    Node* right;
};

typedef Node* address;
#define Nil NULL

address alokasi(infotype x) {
    address P = new Node;
    P->info = x;
    P->left = Nil;
    P->right = Nil;
    return P;
}

void insertNode(address &root, infotype x) {
    if (root == Nil) {
        root = alokasi(x);
    } else if (x < root->info) {
        insertNode(root->left, x);
    } else if (x > root->info) {
        insertNode(root->right, x);
    }
}

address findNode(infotype x, address root) {
    if (root == Nil || root->info == x) {
        return root;
    }
    if (x < root->info) {
        return findNode(x, root->left);
    }
    return findNode(x, root->right);
}

void InOrder(address root) {
    if (root != Nil) {
        InOrder(root->left);
        cout << root->info << " ";
        InOrder(root->right);
    }
}
```

```

    }
}

void PreOrder(address root) {
    if (root != Nil) {
        cout << root->info << " ";
        PreOrder(root->left);
        PreOrder(root->right);
    }
}

void PostOrder(address root) {
    if (root != Nil) {
        PostOrder(root->left);
        PostOrder(root->right);
        cout << root->info << " ";
    }
}

int hitungNode(address root) {
    if (root == Nil) {
        return 0;
    }
    return 1 + hitungNode(root->left) + hitungNode(root->right);
}

int hitungTotal(address root) {
    if (root == Nil) {
        return 0;
    }
    return root->info + hitungTotal(root->left) + hitungTotal(root->right);
}

int hitungKedalaman(address root, int start) {
    if (root == Nil) {
        return start;
    }
    int kiri = hitungKedalaman(root->left, start + 1);
    int kanan = hitungKedalaman(root->right, start + 1);
    return (kiri > kanan) ? kiri : kanan;
}

int main() {
    cout << "Hello World" << endl;

    address root = Nil;

    insertNode(root, 1);
    insertNode(root, 2);
    insertNode(root, 6);

```

```

insertNode(root, 4);
insertNode(root, 5);
insertNode(root, 3);
insertNode(root, 6);
insertNode(root, 7);

cout << "\nInOrder   : ";
InOrder(root);

cout << "\nPreOrder  : ";
PreOrder(root);

cout << "\nPostOrder : ";
PostOrder(root);

cout << "\n\nKedalaman : " << hitungKedalaman(root, 0) << endl;
cout << "Jumlah Node: " << hitungNode(root) << endl;
cout << "Total Info : " << hitungTotal(root) << endl;

return 0;
}

```

Hasil Coding

The screenshot shows a C++ IDE with the following components:

- EXPLORER:** Displays the project structure, including files like `unencoded.cpp` and `unencoded1.cpp`.
- EDITOR:** Shows the source code for `unencoded.cpp`, which includes headers, namespace declarations, and a `Node` struct definition.
- TERMINAL:** Displays the output of the program, showing the results of InOrder, PreOrder, and PostOrder traversals, as well as the depth, node count, and total information.

The output in the terminal is as follows:

```

Microsoft Windows [Version 10.0.26200.7462]
(c) Microsoft Corporation. All rights reserved.

C:\Ryan\Semester 3\IF 07\Struktur Data\MODUL10_103112400081_StrukturData>cd "c:\Ryan\Semester 3\IF 07\Struktur Data\MODUL10_103112400081_StrukturData\output"

c:\Ryan\Semester 3\IF 07\Struktur Data\MODUL10_103112400081_StrukturData\output>.\\unencoded.exe

Hello World

InOrder   : 1 2 3 4 5 6 7
PreOrder  : 1 2 6 4 3 5 7
PostOrder : 3 5 4 7 6 2 1

Kedalaman : 5
Jumlah Node: 7
Total Info : 28

c:\Ryan\Semester 3\IF 07\Struktur Data\MODUL10_103112400081_StrukturData\output>

```

Tujuan program ini adalah membuat program dari bahasa c++ yang mana kita membuat dan mengimplementasikan struktur data Binary Search Tree (BST) menggunakan bahasa pemrograman C++ dengan memanfaatkan linked list (pointer). Program ini dirancang untuk mengelola data bertipe integer melalui beberapa operasi dasar BST,

yaitu penyisipan data (insert), pencarian data (find), serta penelusuran tree (traversal) yang meliputi inorder, preorder, dan postorder.

Selain itu, program ini juga bertujuan untuk menghitung jumlah node, menghitung total nilai data pada seluruh node, serta menentukan kedalaman maksimum (height) dari Binary Search Tree menggunakan pendekatan rekursif. Dengan adanya program ini, diharapkan pengguna dapat memahami konsep kerja BST serta penerapannya dalam pengolahan data secara hierarkis.

Singkatnya sih kita mengimplementasikan Binary Search Tree (BST) dalam bahasa C++ untuk melakukan insert, pencarian, traversal, serta perhitungan jumlah node, total data, dan kedalaman tree.